Chapter 13

# Validating Code with Advanced TypeScript

Over the course of this book, we've been using TypeScript to make assertions about the structure of our code, which has made it easier to validate that our code is correct. Now we're going to focus on features of TypeScript itself that we can use to enhance the typing of our system, with the goal of making it even harder to write code that puts the system in an invalid state.

Many JavaScript programs use a lot of code to protect against invalid data being passed around, continually performing null checks or other type checks. We can use our TypeScript system to make certain kinds of invalid states impossible without using run-time checks. We can, for example, specify that a certain value can't be set to null, and then at compile-time, the compiler must be convinced that a null value can't get there. These techniques are often more verbose than plain JavaScript, but the hope is that the extra typing up front makes the run-time behavior of the code a lot easier to deal with over the long term.

In this chapter, we're going to look at a few different helpful TypeScript features: *union types*, which allow us to create new types by combining existing types; *literal types* and *enums*, which let us limit a type to a set of values; and *mapped types* and *utility types*, which allow us to apply a feature to an existing type. Then we'll explore configuring TypeScript to change the behavior of the compiler itself.